



Projektkonzeption Teil 2

Christina, Daniel, Lea, Lorenzo,
Lukas, Nhi, Andre, Victor, Max

21.09.2026

Agenda

1. Aktueller Entwicklungsstand
2. Deploymentprozess des App Stores
3. Überarbeitete QS & aktualisierte Architektur
4. User Stories
5. Harness Engineering
 - a. Was ist Harness
 - b. Prozesse & Bausteine
 - c. Entscheidungen & Kritik

Identity Provider

Weg	Identity Provider	Mechanismus
Direktzugang App-Store	App-Store Keycloak	OIDC
Moodle → App-Store	Moodle als LTI-Plattform	LTI-1.3-Token
Self-Service-UI → App-Store	DHBW Keycloak (oauth2-proxy)	SSO-Bridge
Immer SSO als E-Mail-Identität		

Refactoring



Auslagerung der Business-Logik

Fehlende API-Routes

Widersprechende “Owner View”

Nicht benutzte API-Routes

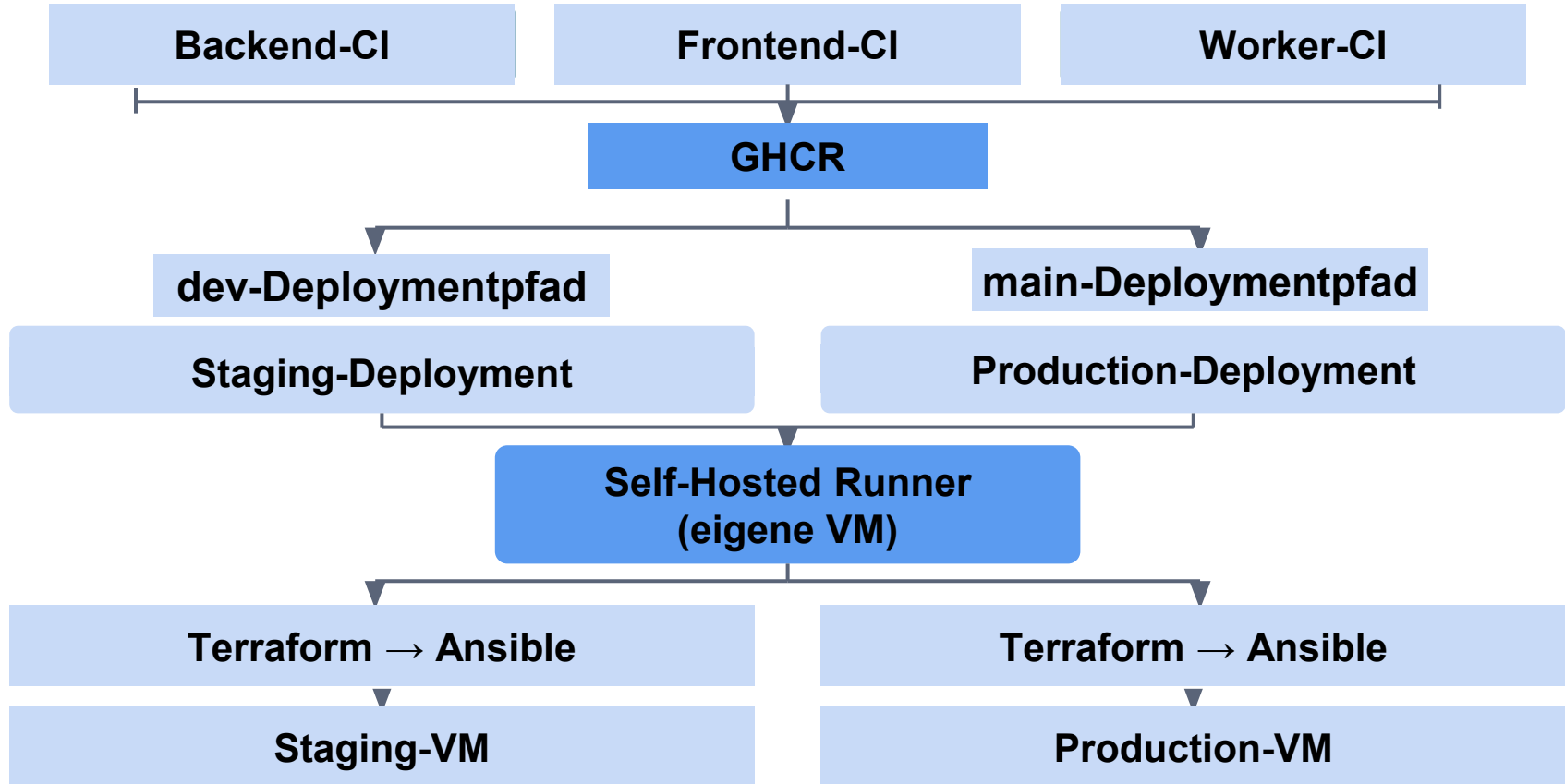
Packer Inkonsistenz

Duplikationen

Frontend Tests

Fehlender Linter im Frontend

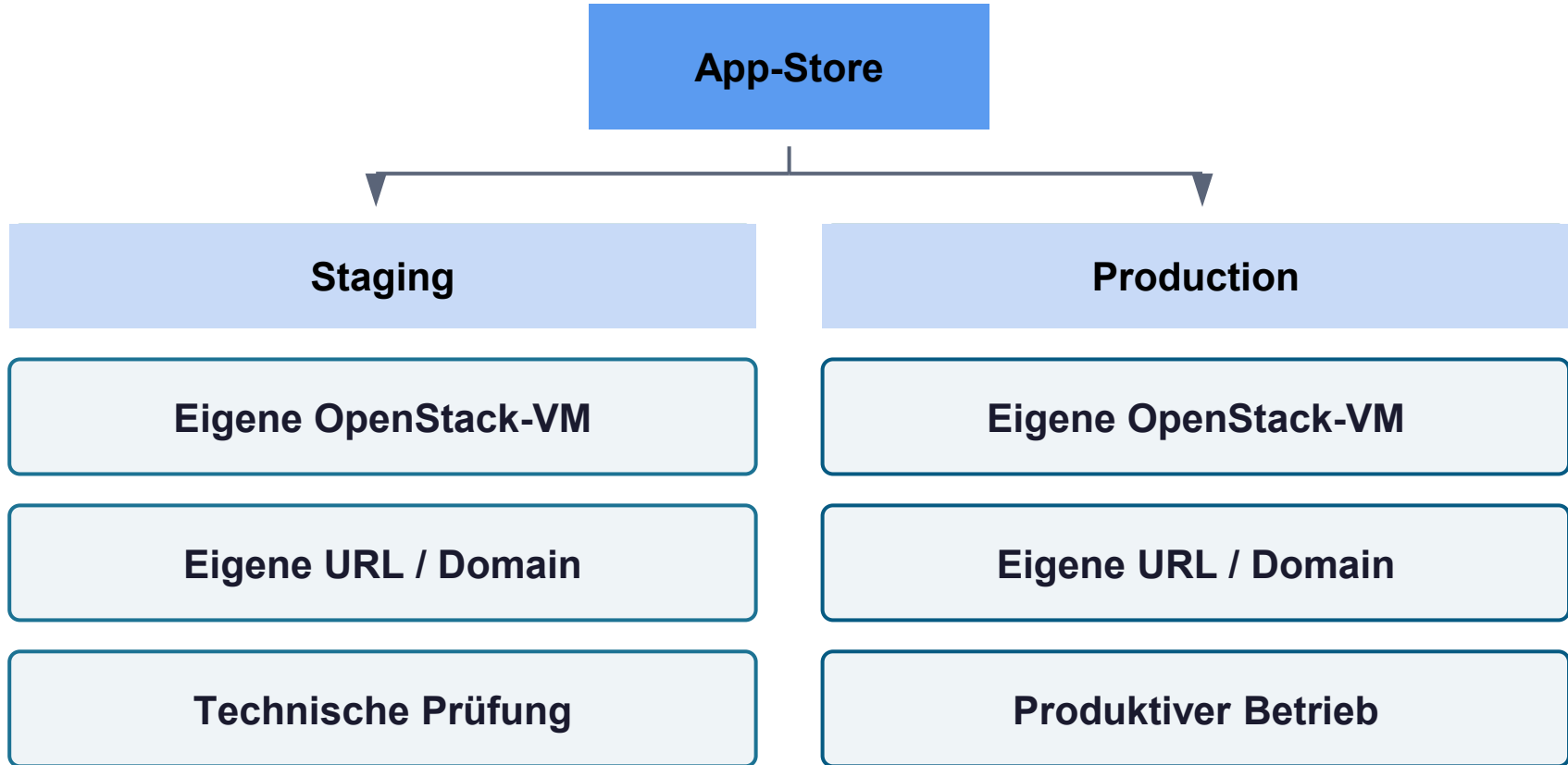
Deploymentprozess des App-Store



Qualitätsbeiträge des Deploymentprozesses

Qualitätsziel	Prozessmaßnahme	Beitrag / reduziertes Risiko
Zuverlässigkeit	Tests + getrenntes Staging	Fehler frühzeitig erkennen
Sicherheit	Security-Scans + TLS	Schwachstellen erkennen & Kommunikation absichern
Reproduzierbarkeit	Terraform + Ansible	Manuelle Konfigurationsfehler & Drift reduzieren
Nachvollziehbarkeit	GitHub Actions + GHCR	Workflow-Läufe und veröffentlichte Images nachvollziehen

Zwei getrennte Umgebungen



P1: Definition of Done

Warum:

- Unterschiedliches Verständnis von “fertig”, Nacharbeit, Konflikte

Wer:

- Definition durch QS-Beauftragte
- Ausführung von allen Teammitgliedern

Wann:

- Review bei Abschluss einer User-Story

Was:

- DoD-Checkliste im Repo
- Abgehakte DoD im Issue

Wie:

- Checkliste im PR-Template, Merge nur bei vollständiger Abhakung

Gut genug als Qualitätsmaßnahme weil:

- Fehler werden vor “Done” gefunden
- Gutes Aufwand/Nutzen-Verhältnis

P9: Moodle-Integration

Warum:

- Drei Systeme (Moodle, LTI, Keycloak),
- Schnittstellen nicht durch Unit-Tests testbar

Wer:

- Zuständig: Backend & Frontend Team

Wann:

- Bei Änderung an LTI-Anbindung,
Moodle-Stand, Keycloak-Konfiguration

Was:

- Protokoll mit Moodle- und LTI-Version

Wie:

- Schritte manuell, in einer Sitzung, in fester Reihenfolge

Gut genug als Qualitätsmaßnahme weil:

- Prüfung gegen real betriebenen Stand, nicht gegen Mock
- Prüfung der korrekten Rollenzuordnung

P12: Dokumentationspflege

Warum:

- Bessere Wartbarkeit
- Missstände in Doku früher erkennen

Wer:

- Pflege: Autor der Pull Request
- Periodischer Abgleich durch QS-Beauftragte

Wann:

- Ereignisgesteuert: bei Änderung an Setup, Ports, Compose-Konfiguration, Deployment, ...
- Periodisch: Gesamtgleich alle zwei Wochen

Was:

- Doku-Abschnitt in Pull Request

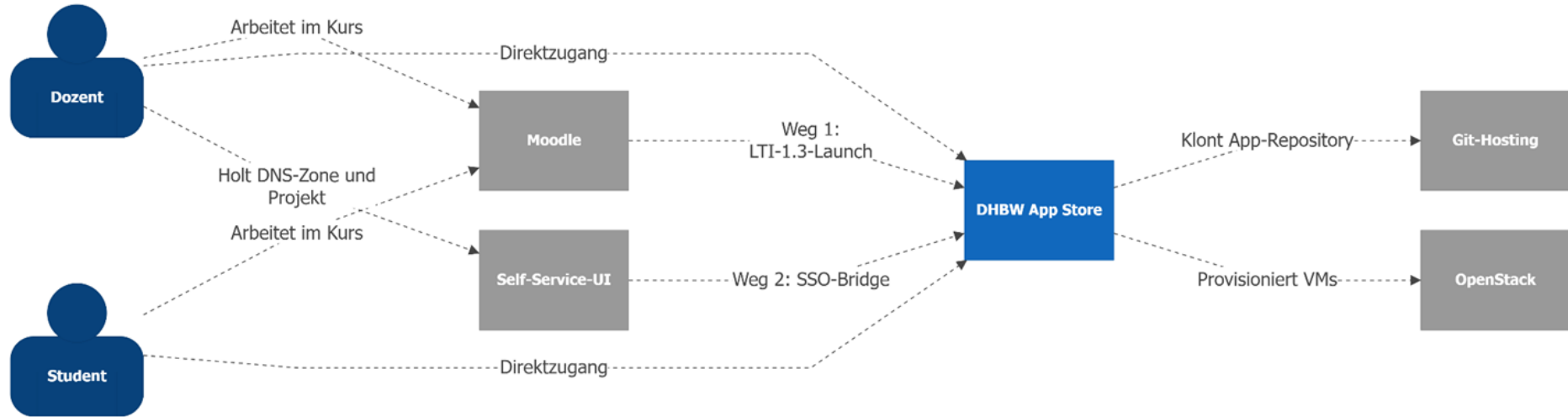
Wie:

- Pull-Request nennt betroffene Zeile der Doku,
- Codeänderung und Dokupflege in selber Pull-Request

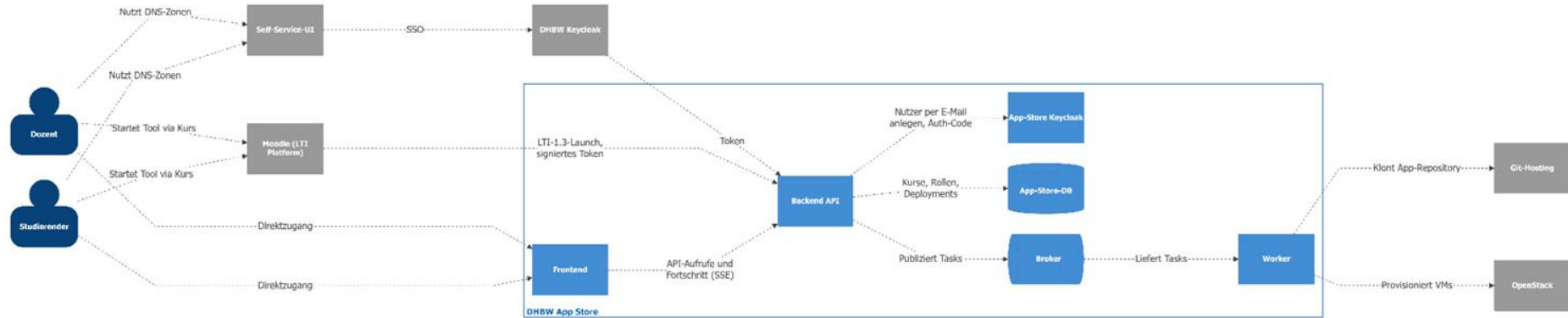
Gut genug als Qualitätsmaßnahme weil:

- Pull-Request verhindert Abweichung der Doku
- Unmittelbare Pflege → ohne Wissensverlust

Authentifizierung und Identitätsauflösung



Aufbau und Deployment-Pfad



USER STORIES · Perspektive Student

Statusanzeige für Studierenden

Story: Als Studierender möchte ich über den Status meiner angeforderten Arbeitsumgebung informiert werden, um genau zu wissen, wann ich mit der Bearbeitung meiner Aufgaben beginnen kann.

Akzeptanzkriterien:

- Eine Status-Anzeige (z.B. "Wird vorbereitet...", "Bereit") ändert sich dynamisch nach dem Start-Klick.

USER STORIES · Perspektive Allgemeiner Dozent

Moodle Integration

Story: Als Dozent möchte ich den AppStore direkt aus Moodle heraus ohne erneute Anmeldung nutzen können, um nahtlos zwischen den Systemen wechseln zu können.

Akzeptanzkriterien:

- Eine Single-Sign-On-Integration mit dem zentralen Hochschul-Identity-Provider ist implementiert (geplant OIDC).

USER STORIES · Perspektive Allgemeiner Dozent

Rollenverteilung

Story: Als Administrator möchte ich durch Nutzerrollen steuern, wer Ressourcen anlegen darf, um unkontrollierten Ressourcenmissbrauch zu verhindern.

Akzeptanzkriterien:

- Das System unterscheidet strikt zwischen Dozenten- und Studenten-Accounts.
- Studierende haben keine Berechtigung und keine sichtbaren Buttons, um eigene VMs aus dem Katalog zu generieren.

USER STORIES · Perspektive Allgemeiner Dozent

Konfigurierung Templates

Story: Als Dozent möchte ich bestehende Standard Templates anpassen und diese optional als "privat" markieren, um spezifische Lernumgebungen exakt für meinen Kurs zu erstellen, ohne den öffentlichen Katalog zu überladen.

Akzeptanzkriterien:

- Vorhandene Templates können dupliziert und inhaltlich editiert werden.
- Ein Sichtbarkeits-Toggle (Öffentlich/Privat) ist für selbst erstellte Templates vorhanden.
- Ein Bestätigungsscreen zeigt eine Vorschau der geänderten Parameter vor dem endgültigen Speichern an.

USER STORIES · Perspektive Datenbanken Dozent

Deployment vorkonfigurierter Datenbank-Server

Story: Als Datenbank Dozent möchte ich eine funktionale Datenbank inklusive grafischem Interface bereitstellen, um Anfängern einen visuellen und einfachen Einstieg in SQL zu ermöglichen.

Akzeptanzkriterien:

- Der Datenbank Server läuft im Hintergrund.
- Ein visuelles Web Interface für die Datenbank Administration ist direkt über Browser aufrufbar und mit Datenbank verknüpft.

USER STORIES · Perspektive Datenbanken Dozent

Vorbereiteter Beispieldatensatz

Story: Als Datenbank Dozent möchte ich, dass die Datenbank mit vorab definierten Datensatz und individuellen Studierenden Accounts startet, damit die Übungen ohne Verzögerung durch Datenimporte oder Rechteprobleme beginnen können.

Akzeptanzkriterien:

- Ein vordefinierter Dummy Datensatz ist beim ersten Start bereits vollständig eingespielt.
- Für jeden teilnehmenden Studierenden wird automatisch separater Account mit eingeschränkten Rechten generiert (keine Admin Rechte).

USER STORIES · Perspektive Advanced IT Dozent

Standardisierte Linux-Entwicklungsumgebung

Story: Als Advanced IT Dozent möchte ich Programmierumgebungen mit vorinstallierten Compilern und IDEs bereitstellen, um direkt mit praktischen Übungen starten zu können.

Akzeptanzkriterien:

- Werkzeuge wie Java, Eclipse, IntelliJ und eine Unix-Shell sind vorinstalliert.
- Notwendige Ausführungsrechte (PATH-Variablen) sind für die Studierenden direkt nach dem Start korrekt gesetzt.

USER STORIES · Perspektive Advanced IT Dozent

Netzwerkverhalten der VMs

Story: Als Advanced IT Dozent möchte ich, dass die Arbeitsumgebungen meiner Studierenden netzwerktechnisch uneingeschränkt untereinander kommunizieren können, um praktische Übungen zu dezentralen Netzwerkprotokollen durchzuführen.

Akzeptanzkriterien:

- Interne Firewalls lassen Broadcast- und UDP-Verkehr zwischen den VMs innerhalb derselben Übungsgruppe automatisch zu, ohne manuelle Portfreischaltungen durch die Nutzer.

USER STORIES · Perspektive Maschinenbau Dozent

Maschinenbau Software

Story: Als Maschinenbau Dozent möchte ich ressourcenintensive CAD-Umgebungen bereitstellen können, damit Studierende komplexe 3D-Baugruppen ruckelfrei und ohne eigenen Installationsaufwand bearbeiten können.

Akzeptanzkriterien:

- Das System weist diesen spezifischen Umgebungen beim Start automatisch hardwarebeschleunigte Profile (z.B. mit vGPU) zu.
- Die erforderliche CAD-Software (z.B. SolidWorks oder Inventor) ist vorinstalliert und direkt startklar.

USER STORIES · Perspektive Maschinenbau Dozent

Matlab

Story: Als Maschinenbau Dozent möchte ich Simulationssoftware inklusive meiner hochgeladenen Skripte lizenzfertig bereitstellen, damit alle Studierenden zum Vorlesungsstart denselben funktionierenden Arbeitsstand haben.

Akzeptanzkriterien:

- Matlab (inklusive benötigter Toolboxes und Hochschullizenz) startet ohne zusätzliche Benutzereingabe.
- Vorab hinterlegte Dozenten-Skripte und Datensätze liegen automatisch im primären Arbeitsverzeichnis der Studierenden bereit.

USER STORIES · Perspektive IT Security Dozent

Bereitstellung von Pentesting Umgebungen

Story: Als IT Security Dozent möchte ich eine Umgebung mit vorkonfigurierten Pentesting Tools bereitstellen, die zwingend in Sandbox läuft, um Angriffsübungen ohne technische Gefahr für produktive Hochschulnetz durchzuführen.

Akzeptanzkriterien:

- Die Umgebung enthält vorinstallierte Werkzeuge wie Kali Linux, Hashcat, John tools und Offline Cracking Tools.
- Netzwerk- und Firewallregeln blockieren technisch jeden Outbound-Traffic aus dieser spezifischen Umgebung in reguläres Campus Netz (Strict Isolation).

Spezifikation rein, geprüfte Lieferung raus

Nach der Spezifikation läuft der Agent eigenständig los, setzt die Anforderung qualitätsgesichert um und bringt sie in den Einsatz – ohne dass die Codebasis degradiert.

DAFÜR BRAUCHT DER AGENT WERKZEUGE ZUM:



Entwickeln



Debuggen



**Qualität
sichern**



Deployen

Ein Harness ist alles am Agenten außer dem Modell



FEEDFORWARD Guides

Steuern zielgerichtet, **bevor** der Agent handelt. Sie geben Arbeitsabläufe, Wissen und Systemkontext vor.

FEEDBACK Sensoren

Prüfen das Ergebnis **danach**, damit der Agent sich selbst korrigiert (z.B. CI-Gates, automatische Linter, Tests).

Quellen: B. Böckeler (2026), „Harness engineering for coding agent users“, martinfowler.com · R. Lopopolo (2026), „Harness engineering: leveraging Codex in an agent-first world“, OpenAI · Formel „Agent = Model + Harness“: LangChain, „The Anatomy of an Agent Harness“

Unser Harness besteht aus vier Ebenen

01 Wissen

Kontext vor Handlung

CLAUDE.md · claude_docs/ ·
HANDOVER.md · Graphify · OpenAPI

02 Skills

Standardisierte Arbeitsabläufe

Brainstorming · TDD · Code Review ·
ship-feature · Ops-Skills

03 Tools

Kontrollierter Zugriff auf Systeme

GitHub MCP · podman-mcp · python-
openstackmcp

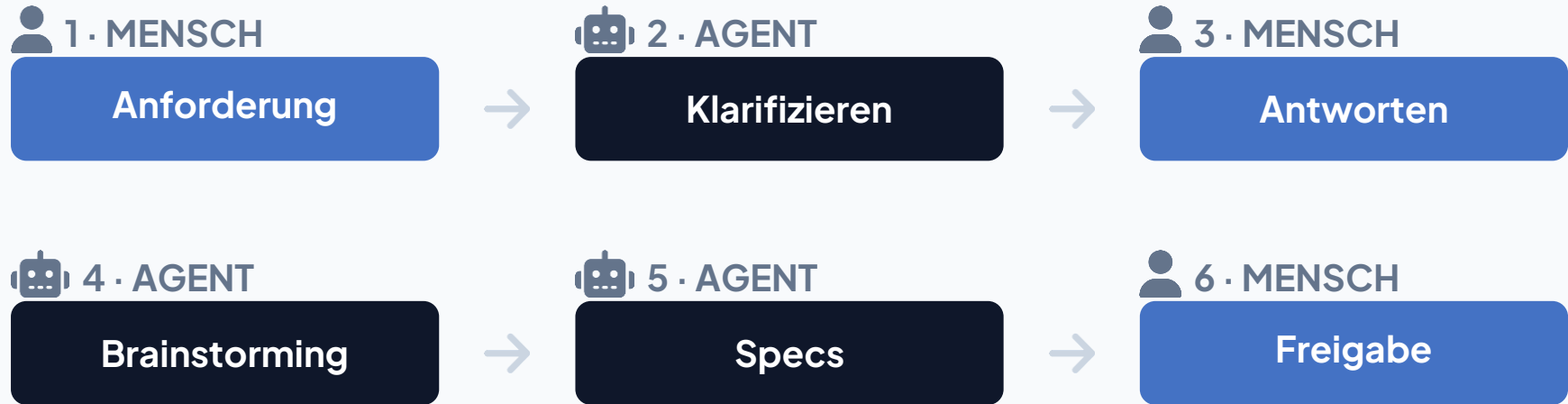
04 Kontrollen

Grenzen für irreversible Aktionen

Branch Protection · CI Gates ·
PreToolUse · HITL

Zwei Prozesse tragen die Anforderung

PROZESS 1 · ISSUE CREATION: Wunsch -> Issue



Prozess 1: Vom Wunsch zur Spezifikation (Issue Creation)



WISSEN

KLARIFIZIEREN & ANTWORTEN

Gesicherter Kontext

Start mit
HANDOVER.md und
claude_docs/: Stand,
Grenzen, bekannte
Fallstricke. Gezielte
Rückfragen, statt
Architektur zu raten.



WISSEN

SKILLS

BRAINSTORMING & SPECS

Blick über alle Repos

Graphify zeigt Abhängigkeiten
zwischen den Repos, OpenAPI
liefert den echten Contract.
Daraus gibt Superpowers
Design Spec Optionen und
Implementation Spec.



KONTROLLEN

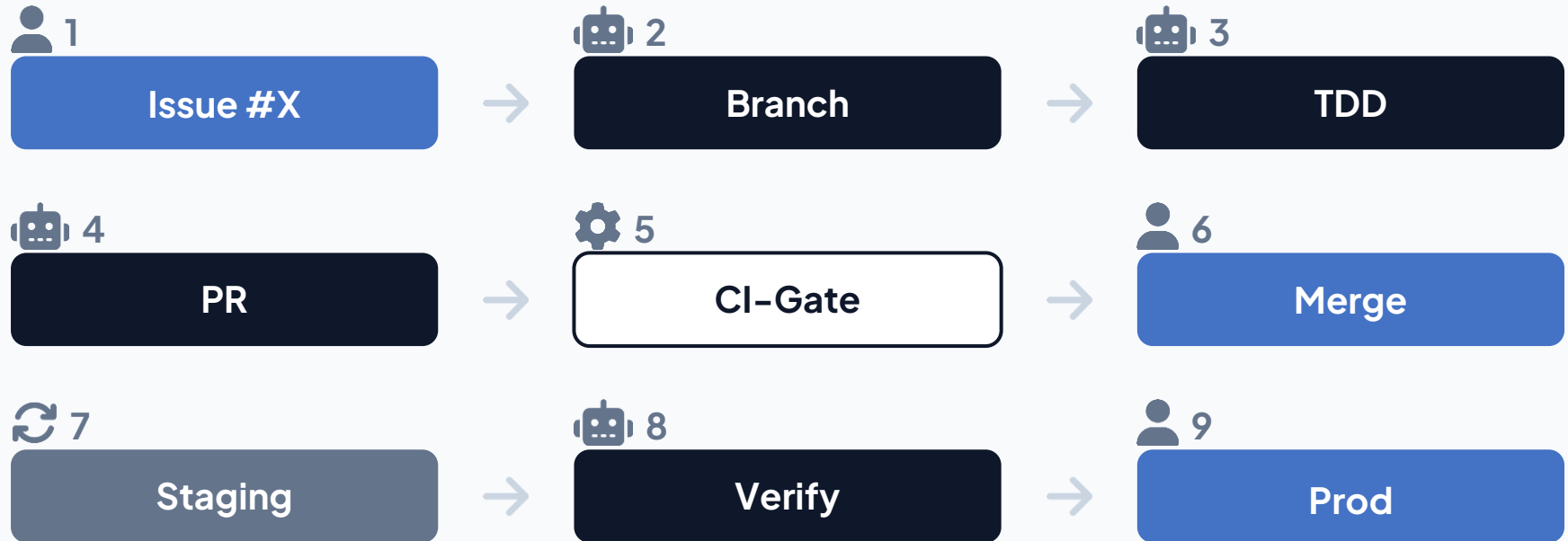
MENSCH, HITL

Menschliches Urteil und Guardrails

Der Mensch formuliert
die Anforderung,
beantwortet
Rückfragen und gibt
die Spezifikation frei.

Zwei Prozesse tragen die Anforderung

PROZESS 2 · DEVELOPMENT & PROD: Issue -> Staging Deployment



Prozess 2: Test Driven Development (Superpowers TDD)

Phase 1

Tests schreiben, die failen

Schreiben von
Tests anhand von
festgelegten
Spezifikation des
Issues.

Phase 2

Minimalen Code schreiben, der die Tests erfüllt

- Ziel: Code, der die Tests erfüllt
- Bei fehlgeschlagenen Tests ist der Code, nicht die Tests zu ändern

Phase 3

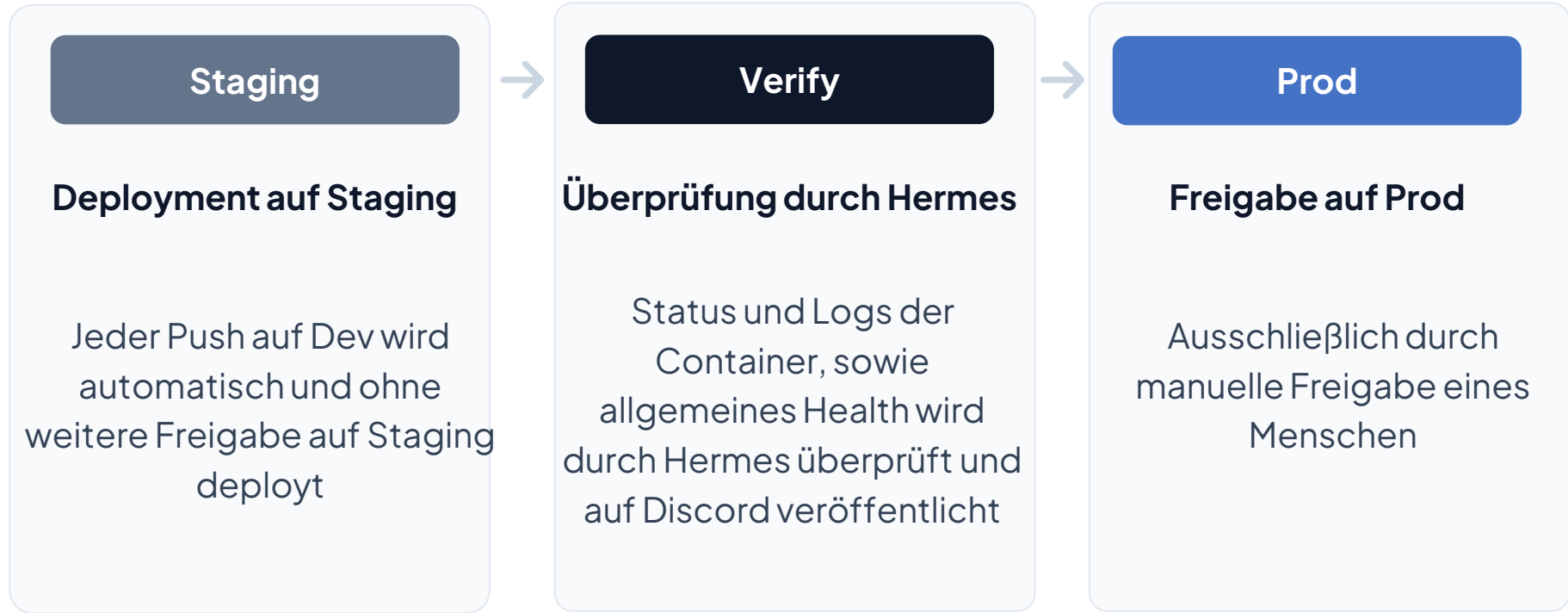
Refactoring und Entfernen von überflüssigem Code

Verwendung von
eindeutigen
Namen und die
Entfernung von
Duplikaten

Prozess 2: CI Gates



Prozess 2: CI Gates



Dokumentation

Zentrale Anweisung

.github enthält Repo-übergreifende Informationen

- Harness.md als Erklärung
- CLAUDE.md als zentrale Anweisung
- Zentrales Graphify als Übersicht

Granulare Anweisungen

Spezifische Doku pro Repo

- CLAUDE.md als Repo-Beschreibung
- HANDOVER.md für aktuelle Änderungen

Schnittstellen

Code selbst als Schnittstelle

FastAPI wird aus vorhandenem Code generiert, so gibt es keine veraltete Doku

Prozess 2: Tools aufgeteilt nach Agenten

Coding Agent (Claude Code)

Wissen: CLAUDE.md, claude_docs/,
HANDOVER.md, Graphify

Skills: /tdd, /ship-feature, /code-reviewer

Tools: gh-CLI für Issue, Branch, PR
und CI-Status

Server-Agent Hermes

Skills: /diagnose-production, deploy-status, restart-service

Tools: Github-MCP, OpenStack-MCP
und Podman-MCP

Eingeschränkter Zugriff: ausschließlich Read-only

Der Agent prüft seine eigene Arbeit



Beweist: es läuft. Nicht: es ist das Richtige.

Coverage misst Zeilen,
nicht Verhalten

Deshalb gibt ein Mensch
die Spezifikation frei

Markdown statt Hooks

CLAUDE.md

„Coverage über 80 %“

Bitte



CI-GATE

Prüft nach

rot → kein Merge

Kein Hook im Lauf

Regeln nur an einer Stelle

Kein Hook, der blockiert

Preis: Feedback erst im CI

Autonom, solange es reversibel ist

Branch

TDD

PR

CI

Staging

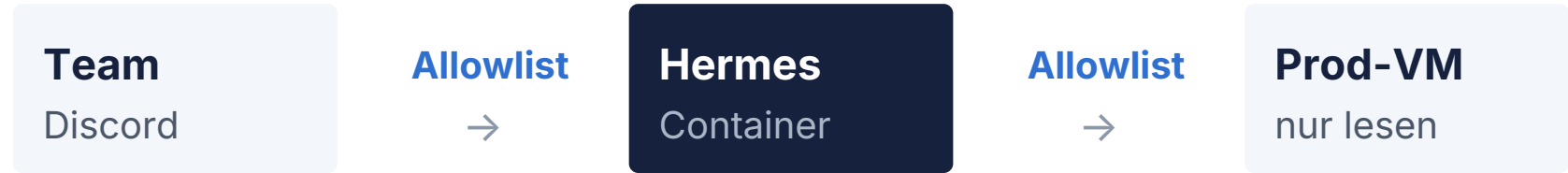
Prod

zurücknehmbar · Agent

Mensch

Staging kann im schlimmsten Fall neu aufgesetzt werden.
Prod muss aber stabil sein.

Zugriff: Hermes statt Agent per SSH



Zwei Filter: **wer reden darf** · **was er tun darf**

Noch Offen

Dev-Agent -> Hermes ? Kommunikation bisher nur über Discord



Danke für eure
Aufmerksamkeit!

Fragen?